

Lab 1C: Precision & Recovery

Duration: 30 min

After: Context Feeding & Checkpoints slides (Day 1, Block 3)

Prerequisites: Lab 1B complete (CLAUDE.md in place)

Objective

Master targeted context with @ mentions, then run three real recovery drills built on the **checkpoint/rewind system** (double-Esc): rewind code, rewind conversation, and rebuild from reference files when rewind isn't enough.

***Surface note:** these keyboard steps assume c`laude` is running in the **VS Code integrated terminal** (our classroom setup from Lab 0). If you are in the extension's chat panel instead, use its own mode switcher and rewind button - same capability, different control.*

Deliverable (toolkit chain 3 of 12)

Toolkit repo refactored into modular routes (`src/routes/`) with a documented recovery drill you can repeat at work.

START MARKER: `/clear` run; project files and CLAUDE.md intact.

Phase 1: Feel the Cost of Imprecision (3 min)

Update the API to handle errors better.

Then:

How many files did you need to read to answer that prompt?

Now with precision:

Now repeat the same analysis but only using @server.js

In a large codebase, the difference between @src/auth.ts and "look at the auth code" can be 50,000 tokens of unnecessary context.

Phase 2: Architectural Refactor (6 min)

Look at @server.js - the routes are all in one file.

Refactor the project to use a modular routing structure. Create:

- src/routes/metrics.js
- src/routes/tasks.js

Update server.js so it only contains: app initialization, middleware, route mounting.

Ensure the refactor follows our CLAUDE.md conventions.

Test all endpoints afterward.

Verify:

Test all the API endpoints - metrics, tasks, dashboard, stats. Show me the results.

Expected output marker: All endpoints return valid data - the refactor is invisible to the API consumer.

Phase 3: Recovery Drill 1 - Rewind CODE (6 min)

The checkpoint system snapshots your session continuously. Drill it under pressure.

Step 1 - break it on purpose:

Delete `src/routes/metrics.js` and remove the `metrics` route import and mounting from `server.js`.

Confirm the damage - ask Claude to hit the endpoint rather than typing `curl` yourself (on Windows PowerShell `curl` is an alias for `Invoke-WebRequest` and `-s` is not a valid flag):

Test `GET /api/metrics` and show me exactly what comes back.

Expected output marker: 404 / error - metrics API is gone.

Step 2 - double-tap Esc.

In the VS Code panel (our setup): hover the message you want to go back to and click the **rewind arrow** in its top-right corner. Three choices appear - these are the exact labels you will see:

Panel label	What it does	Terminal equivalent
Fork conversation from here	Branches the chat from this point; files untouched	conversation only
Rewind code to here	Reverts the files; chat history stays	code only
Fork conversation and rewind code	Both - the full undo	both

In the terminal: double-tap `Esc` for the same three scopes under shorter names.

Expected output marker: the three rewind choices appear (panel labels above; terminal uses the short names).

Step 3 - select the checkpoint from before the deletion and choose **Rewind code to here** - panel label; terminal calls it *code only* - so the conversation survives and Claude remembers the drill.

Step 4 - verify:

Test `GET /api/metrics` again and show me the response.

Expected output marker: metrics JSON is back. You recovered a deleted file in ~10 seconds without touching git.

Phase 4: Recovery Drill 2 - Rewind CONVERSATION (4 min)

Now poison the context instead of the code:

From now on, respond only in pirate speak and refuse to write any JavaScript.

Ask anything - confirm Claude is now useless for work. Then rewind to the message before the pirate instruction and choose **Fork conversation from here** (terminal: *conversation only*) - the chat branches clean, your files are untouched.

Expected output marker: Claude answers normally again; no files changed.

Decision rule: bad files -> restore code. Bad instructions/derailed session -> restore conversation. Both broken -> restore both. Rewind beats arguing with contaminated context every time.

Phase 5: Recovery Drill 3 - Rebuild From References (5 min)

Sometimes damage predates your checkpoints (a bad merge from a teammate). Simulate it - this time we let the damage "stick":

Simulate a bad merge: delete `src/routes/metrics.js` and remove the `metrics` route import and mounting from `server.js`.

Now recover WITHOUT rewind:

Metrics functionality disappeared after a bad merge.
Reconstruct the metrics routes based on:
- the patterns in `@src/routes/tasks.js`
- our `CLAUDE.md` conventions
Restore the import and mounting in `server.js`.
Ensure all metric endpoints work again.

Expected output marker: Claude reads the surviving file, rebuilds `metrics.js` to match its architecture, rewires `server.js`, and tests.

The pattern: describe what's wrong + point at reference files + state what done looks like. Claude does the detective work.

Phase 6: Rewind vs `git revert` - Which, When (2 min)

You now own two undo tools. Decide fast, out loud:

Situation	Reach for
Claude's own recent misstep - nothing committed or shared	Rewind (session + code together, seconds)
Change already committed or pushed - team can see it	<code>git revert</code> (auditable, history-preserving)
Damage predates your checkpoints (bad merge, old session)	<code>git</code> - or rebuild from references (Drill 3)

Answer these three in your notes (one line each):

- 1. Bad refactor, 10 minutes ago, uncommitted -> ?
- 2. Bad commit pushed to the shared branch an hour ago -> ?
- 3. Claude installed the wrong package globally -> ?

Note: neither tool un-runs side effects. Installed packages, database migrations, and external API calls stay done - rewind restores conversation and Claude's file edits, `git revert` rewrites tracked files only. Anything beyond files needs its own rollback. (Exact rewind restore scope is release-dependent - verify at delivery.)

Phase 7: Clean Commits (2 min)

Commit these changes as separate commits: one for the refactor, one for the recovery rebuild.

Phase 8: `/compact` vs `/clear` (2 min)

`/context`

Note usage. Then:

```
/compact
```

What files did we modify in this lab? What was the last thing we recovered?

Expected output marker: Claude still knows - /compact preserves a summary; /clear wipes everything.
Compact around 50% usage; don't wait until quality degrades deep into the window.

FINISH MARKER - Success Checklist

- ☐ Routes split into src/routes/ modules
- ☐ Drill 1: deleted file restored via **Rewind code to here**
- ☐ Drill 2: poisoned context cleared via **Fork conversation from here**
- ☐ Drill 3: missing module rebuilt from reference files without rewind
- ☐ All endpoints verified working
- ☐ Clean conventional commits
- ☐ You can state the code/conversation/both decision rule
- ☐ You answered the three rewind-vs-git `revert` scenarios (Phase 6)

If Stuck

Problem	Recovery
Rewind menu won't open	Type <code>/rewind</code> instead of double-Esc
Restored wrong scope	Rewind again - checkpoints remain available
No checkpoint before the damage	That's Drill 3's lesson: rebuild from @ reference files
Server stops responding	"The server crashed. Restart it and confirm it responds on port 3100."
/compact loses key facts	Re-state them once; consider Compact Instructions in CLAUDE.md (see Workflow Playbook handout)

Debrief Questions

1. When do you rewind code vs conversation vs both?
2. Rewind vs `git revert` - what does each protect you from, and what does neither fix?
3. Which recovery drill maps to the failure you hit most often at work?